Local and Global Whole-Body Retargeting for MABEL:
Two Modes Separated by the Admitted Measurement Set,
and What That Separation Costs
(`mabel.retargeting`)

MABEL Project — Control & Teleoperation

July 28, 2026

Abstract

MABEL's whole-body teleoperation has, until now, had a single mode in which the operator's body pose is interpreted in the headset's world anchor frame $\mathcal{F}_a$ and the operator's walking and turning drive the mobile base. That mode fails in a moving vehicle for a reason that is invisible from the inside: the cabin is visually static, so tracking looks healthy, while the inertial unit senses the vehicle's specific force, so the anchor's heading integrates the vehicle's yaw and its gravity reference tilts under lateral acceleration. This document describes the two-mode retargeter that replaces it. Following §V-B.1 of the MABEL whole-body control and retargeting paper, the modes are distinguished by the *admitted measurement set* rather than by which outputs are enabled: LOCAL admits only device-frame optical measurements and holds the base, GLOBAL additionally admits the device pose in $\mathcal{F}_a$ and lets body motion command the base. They meet downstream at exactly one rotation and one base channel, so everything else is mode-invariant by construction, and the handoff between them is *exact* rather than merely smooth. We report what works — machine-precision invariance to an arbitrary anchor drift, an exact mode handoff, a drift-free neck — and, at equal length, what does not: the local torso estimator's only evidence about head-versus-torso rotation is the operator's own hands, and we show by measurement that this forces a trade between immunity to the vehicle and steadiness of the reach frame under free head motion, with no filter able to split the difference.

Keywords: teleoperation, whole-body retargeting, reference frames, visual-inertial drift, mobile manipulation, Vision Pro, robust estimation.

1 Overview

1.1 The problem

The operator wears a headset; the robot is elsewhere. Retargeting maps the operator's body onto the robot's, and every such map needs a *body frame* — an answer to "which way is the operator facing?" — because the robot must reproduce the operator's motion *relative to their own torso*, not relative to the room.

MABEL's shipped stack answers that question in $\mathcal{F}_a$, ARKit's world anchor frame: the operator's heading is integrated there, and their head displacement in $\mathcal{F}_a$ drives the base, torso and lift. Three properties of the shipped code follow from that choice and all three break in a vehicle:

1. the shoulder estimator's "up" is $\mathcal{F}_a$'s $+Y$, i.e. the visual-inertial gravity estimate;
2. the base tracker's reference is a latched $\mathcal{F}_a$ head position, and the lift follows $\mathcal{F}_a$ head *height*;
3. the neck's reference is a latched $\mathcal{F}_a$ head *orientation*.

A car corrupts all three at once. Worse, it does so without any symptom the operator can see: ARKit's visual features are the cabin, which is stationary relative to them, so tracking quality indicators stay green.

A second, smaller problem is that MABEL had no way to ask for "upper body only". The wire's `region` field (`arm/upper_body/whole_body`) parameterises the *torque* engine and has never gated the retargeting path: in retarget mode the base followed the head regardless.

1.2 The idea

Rather than adding switches to disable outputs, we partition the measurement stream. Write T_{ad} for the device pose in the anchor frame and T_{ah_i} for a hand pose in the same frame; then

$$T_{dh_i} \triangleq T_{ad}^{-1} T_{ah_i} \tag{1}$$

is that hand expressed in the *device* frame $\mathcal{F}_d$. Both factors are referenced to the same anchor at the same instant, so whatever $\mathcal{F}_a$ is doing cancels *exactly* in the product: T_{dh_i} is a pure optical measurement of where the hand is relative to the headset. The two admitted sets are then

$$\mathcal{S}_{\mathrm{L}} = \{T_{dh_{\mathrm{L}}},\, T_{dh_{\mathrm{R}}},\, \text{skel.}\}, \qquad \mathcal{S}_{\mathrm{G}} = \mathcal{S}_{\mathrm{L}} \cup \{T_{ad}\}. \tag{2}$$

Two consequences are worth stating immediately. First, **no client change is required for the geometry**: (1) is computed server-side from what every MABEL frontend already streams, so the app only has to say *which mode*. Second, the separation is enforceable in code rather than by discipline — the local estimator's `update` takes a `DeviceMeasurements` and has no parameter through which T_{ad} could arrive, which is what makes the invariance claim in §3.1 a checkable property instead of an assertion about control flow.

1.3 Scope and honesty

Everything measured here is measured against a *synthetic* operator. That is a deliberate choice, not a shortcut: validating the vehicle case requires superposing a repeatable anchor drift on a repeatable bimanual task, and no physical rig shakes a headset like a car while a human performs an identical reach twice. What the experiments validate is the frame algebra and the estimator — which is precisely where the mode split lives. Nothing here is a claim about hardware; §5 lists what remains to be verified on `thor`.

2 Method

2.1 Frames and notation

$\mathcal{F}_a$ is ARKit's world anchor frame (X right, Y up, $-Z$ forward), $\mathcal{F}_d$ the device (headset) frame, and $\mathcal{F}_b$ the robot world frame (x forward, y left, z up). A *body basis* B is a 3×3 matrix whose columns are (right, up, forward) expressed in some source frame, so body coordinates of a source-frame vector v are $B^{\top} v$. This is the same convention as `whole_body_retargeting.calibration.body_frame` and `BodyFrameEstimator.frame()`, so the three interoperate.

2.2 Operator model

The operator is three rigid quantities in a torso frame whose origin is the mid-shoulder root: shoulder centres $r^{\sigma_i, o} = (\pm w/2, 0, 0)$, the device (eye) position p_d^o, and per-side arm length L_o.

A measured `OperatorProfile` supplies them when the operator has calibrated; otherwise they are estimated live from Winter's segment ratios of a stature that is itself estimated in $\mathcal{F}_d$ — from the median forearm length $\|\text{wrist} - \text{elbow}\|$ when the skeleton carries an elbow, and otherwise from the running maximum of $\|\text{wrist} - \text{device}\|$. The forearm path matters: it is a rigid segment and so is observable at *every* arm configuration, whereas the reach maximum is only correct once the operator has fully extended, and under-sizes them until they do (measured: a 1.75 m operator who never exceeds $\rho = 0.6$ is sized at 1.51 m).

2.3 Local mode: the torso fit

With no $\mathcal{F}_a$-referenced quantity admitted, the body frame must be estimated directly in $\mathcal{F}_d$. The operator's shoulders are rigid on the torso and each wrist lies within arm's length of its own shoulder, which makes both wrists evidence about torso placement:

$$\hat{T}_{do} = \operatorname*{argmin}_{T \in SE(3)} \sum_i \varrho\Big(\big\| r_d^{h_i,d} - T\, r^{\sigma_i,o} \big\| - L_o \Big)^2 + \mu \big\| \mathrm{Log}\big((T^-)^{-1} T \big) \big\|^2, \qquad \text{s.t. } T \in \mathcal{T}_{\text{neck}}. \tag{3}$$

It is a 3-DOF problem. $\mathcal{T}_{\text{neck}}$ says the torso is reachable from the device through the cervical range. Because the device's own torso coordinates p_d^o are part of the operator model, the torso origin in device coordinates is pinned at $t_{do} = -R_{do}\, p_d^o$, and a shoulder lands at

$$s_i^d = R_{do}\, c_i, \qquad c_i \triangleq r^{\sigma_i,o} - p_d^o, \tag{4}$$

a pure rotation of a known constant. The translation cancels, only the rotation is free, and the cervical box is imposed *structurally* rather than as a penalty. We solve with damped Gauss–Newton on the $\mathfrak{so}(3)$ tangent, $R \leftarrow R \exp(\delta)$, using the analytic row

$$u_i = p_i - R c_i, \quad d_i = \|u_i\|, \quad e_i = d_i - L_i, \qquad J_i = \left(\frac{u_i}{d_i} \right)^\top R\, [c_i]_\times, \tag{5}$$

and the normal equations $(J^\top W J + \mu I)\, \delta = -(J^\top W e + \mu \Delta)$ with Δ the accumulated tangent step from the previous frame's estimate. (5) agrees with central differences to 6×10^{-11}.

Two departures from the literal residual, both forced by measurement. Read literally, (3) constrains each *wrist* to lie *at* arm's length, which is only true when the arm is straight. At a comfortable $\rho = 0.6$ reach the residual sits at -0.14 m and no rotation can null it, so a two-sided quadratic rotates the torso purely to chase an infeasible target. We therefore (i) treat the wrist term as the *inequality* it really is, $d_i \le L_o$, giving violations full weight and the feasible side a small weight w_{slack}, which is what the robust loss ϱ is for; and (ii) add the *elbow* when the skeleton carries one, since $\|\text{elbow}_i - \text{shoulder}_i\|$ equals the upper-arm length exactly at every arm configuration. ARKit's `forearm_arm` is extrapolated rather than tracked, which is why the arm IK ignores it as a *pose* objective, but a segment *length* tolerates that far better. Fig. 2 quantifies what the elbow buys.

Regularisation and its scale. μ balances *metre* residuals against *radian* steps; the damped step is a fraction $H/(H+\mu)$ of the full step, so $\tau \approx \Delta t / k$. With the measured data curvature $H \sim 0.1\,\text{m}^2\,\text{rad}^{-2}$ this fixes μ near 10^{-2}, and μ is rate-normalised ($\mu\, \Delta t_{\text{ref}} / \Delta t$) so the time constant is in seconds rather than frames — the apps stream at 30 Hz and the simulator substeps faster. A first draft used $\mu = 40$, some 400× too stiff, which froze the fit at its seed regardless of what the

operator did. Two elbow spheres are only two constraints on three unknowns, so one direction is always unobserved; a small yaw-invariant prior on the torso's up axis shapes it (a statement about the *neck*, not about gravity, and therefore legal in a vehicle).

Non-convexity. (3) is not convex in the torso rotation and Gauss–Newton is local: from the identity seed alone a genuine 30° body turn converged 25° away and the continuity prior then pinned it there for the session. The cold start therefore multi-starts over yaw and pitch. Roll is deliberately *not* swept: measured over 13 neck poses, yaw-only seeding gives 4.4° mean / 28.0° max, yaw×pitch 3.1° / 9.0°, and yaw×pitch×roll *worsens* to 6.0° / 50.7° — more search finds lower cost and a worse answer, because at a range-of-motion corner the objective's global minimum is no longer the truth. That is a property of the formulation, not of the optimiser, and it is why the response is a bounded sweep plus an honest confidence output rather than a better solver.

2.4 Global mode

Global mode integrates heading in $\mathcal{F}_a$ from the yaw shared by both wrists and the head,

$$\Delta\psi_o = (1-\chi)\,\Delta\psi_{\text{rig}} + \chi\,\psi_\beta\,\gamma(\Delta\psi_d, \psi_\beta), \tag{6}$$

where γ scores consistency with head motion. This is *not* reimplemented here: the bearing and wrist-orientation common modes, the parallel-sweep ambiguity χ and the head-corroboration gate already exist, measured and validated, in `whole_body_retargeting.body_frame_estimator`. `GlobalBodyFrame` is a thin adapter that re-expresses that estimator's output in this package's vocabulary and is the only place $\mathcal{F}_a$ poses are re-composed from $\mathcal{F}_d$ ($T_{ah} = T_{ad}T_{dh}$), keeping the $\mathcal{S}_{\text{G}} \setminus \mathcal{S}_{\text{L}}$ dependency in one readable place.

2.5 Arbitration, and why the handoff is exact

The entire mode-dependent surface is two quantities:

$$R_{bo} = \begin{cases} R_{ba}\hat{R}_{ao}, & m = \text{G} \\ R_{b\star}\hat{R}_{do}, & m = \text{L} \end{cases} \qquad \nu_b^{\text{cmd}} = \begin{cases} \nu_b^{\text{ret}}, & m = \text{G} \\ 0, & m = \text{L}. \end{cases} \tag{7}$$

The two estimators express the body basis in *different* frames, so a naive switch would jump the targets by whatever they disagree about. Writing each mode's map into operator-body coordinates and referring both to $\mathcal{F}_a$ (a physical vector satisfies $v_d = R_{ad}^\top v_a$),

$$P_{\text{G}} = A_{\text{G}}B_{\text{G}}^\top, \qquad P_{\text{L}} = A_{\text{L}}B_{\text{L}}^\top R_{ad}^\top, \tag{8}$$

continuity of the *targets* is $P_{\text{new}} = P_{\text{old}}$ at the switch instant, which has a closed form (B orthonormal, so $(B^\top)^{-1} = B$):

$$\text{G}\to\text{L}\colon\ A_{\text{L}} = P_{\text{G}}R_{ad}B_{\text{L}}, \qquad \text{L}\to\text{G}\colon\ A_{\text{G}} = P_{\text{L}}B_{\text{G}}. \tag{9}$$

The latch reads T_{ad} *once*, purely to relate two bases; local mode never touches it again. If the operator is switching to local because $\mathcal{F}_a$ has gone bad, that corruption enters only as a fixed offset in the mapping's zero — which is the semantics an operator expects from "re-anchor here", and strictly better than a jump.

One subtlety cost real debugging time and is worth recording: the alignment constant corrects the body *basis*, and the shoulders and root are rigid consequences of the basis, so it must be pushed

through the whole estimate, not just through R_{bo}. Correcting only the map leaves the anchors where the raw estimator put them, and the result is a continuous rotation with a 30 cm target jump — a failure that looks like a bug in the latch and is not.

Both estimators run every frame in both modes. That costs little and buys three things: a switch takes effect on the next frame with no warm-up, the latch always has both bases for the *same* instant, and the inactive estimator's diagnostics stay available. It does not weaken the invariance argument, which is about what drives the *output*.

2.6 Downstream: mode-invariant by construction

Wrist targets consume only R_{bo}:

$$\rho_i = \min(\|p_i - s_i\|/L_o,\, 1)\,, \qquad \sigma = L_b/L_o, \tag{10}$$

$$r_i^{\text{ext}} = r_b^{\sigma_i b} + \rho_i L_b R_{bo} u_i, \qquad r_i^{\text{shp}} = \mu_b + \sigma R_{bo}(p_i - \mu_o), \tag{11}$$

$$r_i^* = (1-\beta_i)\, r_i^{\text{shp}} + \beta_i\, r_i^{\text{ext}}, \qquad \beta_i = S(\rho_i; 0.85, 1). \tag{12}$$

Anchoring at each shoulder preserves *reach* exactly but distorts bimanual *shape*, because MABEL's arm mounts are 0.148 m apart against a human's ∼0.45 m; a single similarity about the mid-shoulder root preserves shape but truncates reach. Which matters depends on ρ, and β_i is a smoothstep so the blend is C^1. Finger retargeting is untouched and needs no mode awareness at all: hand skeletons are wrist-referenced and hence already device-frame.

Lift and torso are driven in local mode by the comfort region on the hand centroid $c = \frac{1}{2}(r_{\text{L}}^* + r_{\text{R}}^*)$, $u_c = K_c([c - c^+]_+ - [c^- - c]_+)$ — a robot-frame quantity, so it needs no world reference. (The paper prints the two positive parts with a +; taken literally that drives the same direction on both sides of the box, so we implement the signed deadband, the only reading that restores the hands toward the region from either side.) Global retains the existing head-height lift law by product decision, and the arbiter emits a cross-fade weight so a mode change blends the two laws rather than stepping between them.

3 Experiments

All figures are regenerated from scratch by `controller/experiments/exp_retarget_modes.py`, which is deterministic and headless and prints the summary table every number below is taken from.

3.1 Invariance to the world estimate

An identical operator performs an identical bimanual task twice — once with an anchor drift superposed, modelling the vehicle: heading integrating at 5 to 45° s^{-1}, a gravity-reference tilt of 2 to 12° oscillating with the vehicle's weave, and a translational wander. Fig. 1 reports the resulting wrist-target error against the undrifted baseline.

Local mode's worst-case error over every condition is 1.1×10^{-12} mm: not small, but *machine precision* — the anchor cancels algebraically, as (1) says it must. Global mode's error grows with the drift, to 86 mm peak under the harsh condition. The contrast is the point: the two rows must differ by twelve orders of magnitude, and a regression that quietly leaks an $\mathcal{F}_a$ quantity into the local path shows up here immediately.

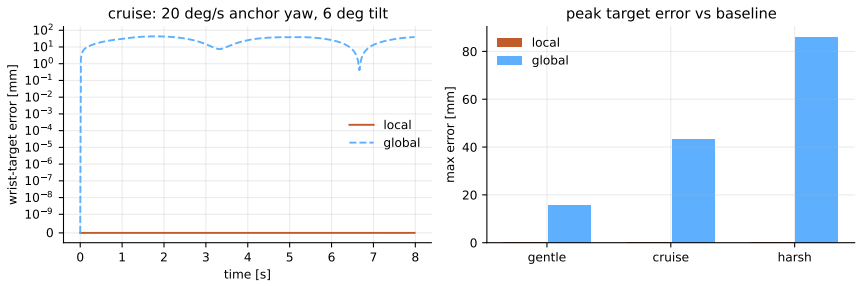


Figure 1: Wrist-target error against a no-vehicle baseline. Local mode is invariant to machine precision (note the symmetric-log axis); global mode degrades with the anchor drift.

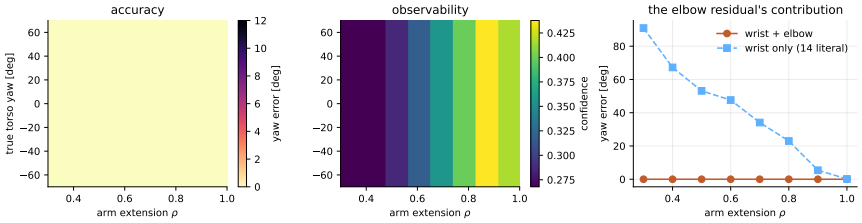


Figure 2: Accuracy and observability of (3) across the workspace, and the contribution of the rigid elbow residual.

3.2 Conditioning of the torso fit

Fig. 2 sweeps arm extension ρ against true torso yaw, reporting the recovered yaw error and the observability measure (the smallest singular value inside the weighted Jacobian's row space, normalised by the shoulder lever, so it reads as "what fraction of a rigid lever arm is observable"). With the elbow residual present the fit is essentially exact across the sweep and the observability never approaches its floor. The third panel is the ablation that justifies the departure from the literal residual: the wrist-only form of (3) carries a 40° mean yaw error over the same sweep.

3.3 The handoff

Fig. 3 switches mode mid-motion in both directions. The target discontinuity at the switch is 1.8 mm, against 1.8 mm of ordinary task motion in the same frame — i.e. the switch is indistinguishable from not switching, which is what (9) promises. The base is ramped rather than cut, and settles to exactly zero.

3.4 Fidelity, and the neck

Fig. 4 reports extension fidelity ($|\rho_{\text{robot}} - \rho_{\text{human}}|$) and bimanual shape fidelity across the workspace. Fig. 5 is the clearest single argument for local mode's estimator: under a $20\,^\circ\,\text{s}^{-1}$ anchor heading drift, the neck command derived from $\hat{R}_{do}^{-1}$ accumulates 0.15° of error over 10 s while the world-anchored reference the engine uses today accumulates 160°. Gaze survives in local mode, and is in fact *better* there.

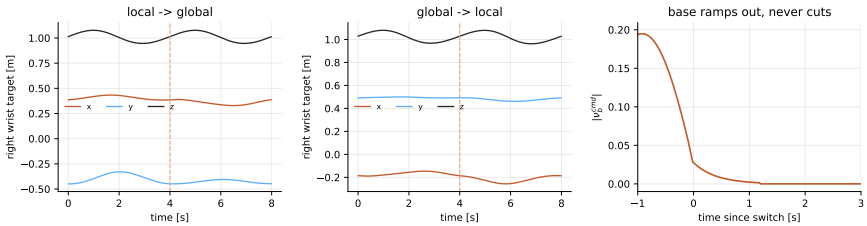


Figure 3: Wrist targets through a mode switch (dashed line), and the base channel leaving on a ramp.

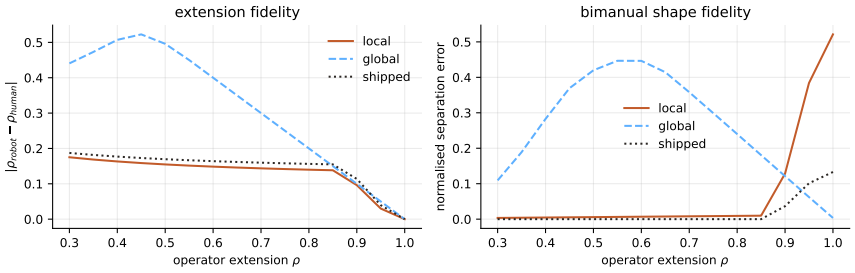


Figure 4: Extension and bimanual-shape fidelity of (12).

3.5 The cost: uncommanded motion under free head movement

The reach frame is only as steady as the torso fit, and the fit's only evidence about head-versus-torso rotation is the operator's hands. So if the operator holds their hands still relative to their torso and moves only their head, any motion of the robot's targets is *uncommanded*. Fig. 6 measures it.

Table 1: Uncommanded target motion under pure neck motion, hands fixed relative to the torso, and the corresponding invariance. Dead-reckoning the head removes the wobble entirely and destroys the invariance.

	paper-literal	dead-reckoned
head still	0.0 mm	0.0 mm
look-around (±60° yaw, 0.2 Hz)	2.9 mm	0.0 mm
nod (±30° pitch, 0.15 Hz)	174 mm	0.0 mm
both at once	176 mm	0.0 mm
error under a 20 °s^{-1} vehicle	0.0000 mm	203.6 mm

The nod row is the honest weak spot, and it is not lag: it is invariant to μ. It is the price of pulling the torso's up axis toward the device's, which is what buys the excellent look-around row; the opposite choice (pure tilt continuity) trades them, giving 2 mm on the nod and 136 mm on the look-around. Glancing is the head motion operators make constantly and a nod reads as a slow vertical drift rather than a jump, so the default takes that side of the trade.

The right-hand column is the finding that matters. Propagating R_{do} through the device's frame-to-frame rotation — the obvious sensor-fusion answer — zeroes *every* wobble row and turns an exact 0.0000 mm into 203.6 mm of uncommanded error in a turning vehicle. And no filter splits the difference, because the corrupting motion (vehicle yaw, ∼20 °s^{-1}) and the legitimate motion

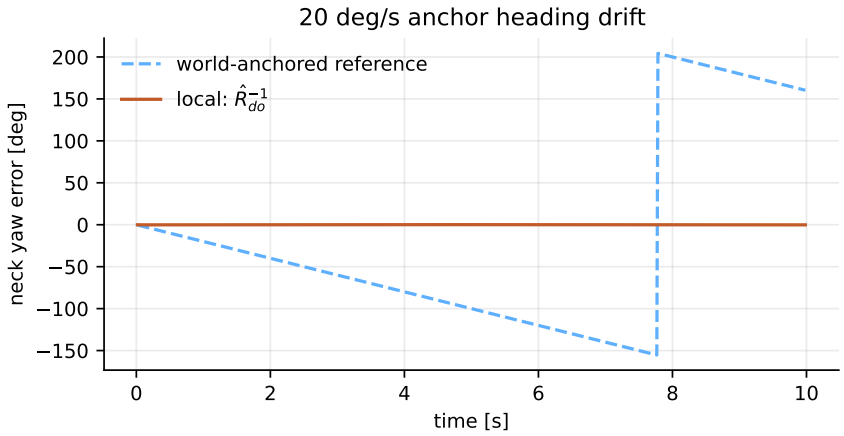


Figure 5: Neck-yaw error under a drifting anchor heading. The local estimate is drift-free by construction; the world-anchored reference inherits the drift one-for-one.

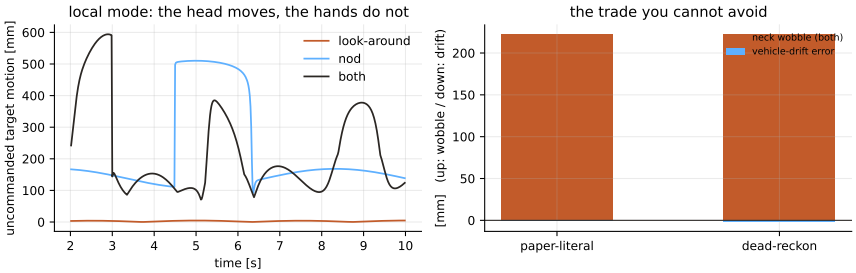


Figure 6: Uncommanded target motion while only the head moves, and the trade against vehicle invariance.

(head yaw, $\sim 75\,^\circ\,\mathrm{s}^{-1}$) are the same order on the same sensor: there is no frequency separation to exploit. Breaking the tie requires more information about the torso than two hands carry.

3.6 What the shipped engine did, measured over the wire — and its removal

Everything above characterises *this package*, which is an offline study and remains intact. The engine the apps actually talk to (`controller/mabel/teleop_engine`) also carried a mode split for two weeks, on the operator-validated GMR arm map, selectable from the Vision Pro by a LOCAL | GLOBAL segment. **That live path was removed on 2026-07-28 at the operator's direction; the engine now runs the global law only.** This subsection is the record of what it measured while it existed, kept because the numbers are the evidence for the design claims above and because the removal was a product decision, not a refutation.

The measurement was one scripted operator motion — a 0.45 m side-step with a 25° body turn, while both hands reach 0.28 m forward and 0.22 m up — replayed identically in both modes against a live `server/run.sh`, and then replayed again with a phantom $9\,^\circ\,\mathrm{s}^{-1}$ yaw and $9\,\mathrm{cm\,s}^{-1}$ translation applied rigidly to the whole operator (the drift a vehicle injects into $\mathcal{F}_a$). The simulation was reset before each run; without that, each run starts from where the last left the robot and the base-travel figure varies by 40 %.

Two deviations from the theory above were worth stating plainly, and both outlived the mode.

	still world		change under drift	
	global	local	global	local
base travel [cm]	33.0	**0.01**	48.5	**0.01**
base yaw [deg]	24.6	**0.02**	50.1	**0.01**
lift [cm]	0.02	15.3	0.34	**0.01**
torso [deg]	0.02	−15.1	22.8	**0.00**
arms [deg]	89.4	104.0	27.0	**3.3**
neck [deg]	24.8	24.8	54.8	54.9

Table 2: The live mode split as it stood on 2026-07-28, measured over `ws://...:9090/teleop` immediately before the local path was removed. Local held the base outright and drove the whole upper body; under vehicle drift its base, lift, torso and arms were invariant and the neck was not. Reproducing this table requires the removed code and its harness (`server/tools/exp_retarget_frame.py`, deleted with it); the numbers stand as a dated record, not as a runnable check.

The engine did not use eq. (20)'s comfort region for lift and torso, and should not have. That law assumes a target that does not move with the actuator driving it. The shipped arm map anchors each wrist target at the robot's mid arm-mount root, and that root rides both the lift and the torso, so raising the lift raises the target by exactly as much: the error never closes. Wired literally it is positive feedback, and it ran the lift to 52.9 cm of the 63.5 cm available and pinned the torso at its −60° stop within 5 s. The replacement was a static map from an anchored reference — the operator's hand centroid relative to their own head, expressed in the head's frame, which is device-frame and therefore carries the same invariance — bounded by the operator's arm length whatever the robot does. That is the 0.01 cm and 0.00° in Table 2.

The neck inherited the drift in both modes, and that is now fixed. Its reference was a latched $\mathcal{F}_a$ head orientation in *both* modes, and the measured deviation was identical in the two (54.8° against 54.9°) — the signature of a shared reference rather than a mode-specific leak. `LocalTorsoFit.cervical_angles` is the principled fix and is validated in Fig. 5 (0.15° over 10 s), but it is not a per-frame drop-in on the control thread: measured 2.4 ms median, 14 ms p95 and 100 ms max — the periodic basin re-check — against a 33 ms budget, and attaching it collapsed the bridge's retarget rate to 1 Hz to 9 Hz. The shipped engine instead takes the neck relative to the *torso axes the arm map already computes every frame* (`ShoulderEstimator.body_axes`), which costs nothing, is drift-free by construction, and is re-zeroed by the operator's Set Forward button. The world-anchored orientation survives only as the fallback for the frames before the estimator has axes.

3.7 Cost per tick

The local fit costs 1.23 ms, the global estimator 0.48 ms, and a full tick 1.90 ms on the development Mac. The global estimator already ran before this package, so the *added* cost is ∼1.4 ms per frame — about 4 % of the budget at the apps' 30 Hz stream rate, and comparable to the existing arm solver's 0.3 ms to 0.6 ms per side.

4 Implementation and usage

4.1 Layout

`controller/mabel/retargeting/` holds the whole method and imports no MuJoCo, so it is unit-testable headless: `types.py` (the admitted sets), `frames.py` ((1) and the $SO(3)$ helpers), `operator_model.py`, `local_frame.py` ((3)), `global_frame.py` (the adapter), `mode.py` ((7) and (9)), `wrist_targets.py` ((12)), `body_dofs.py`, `retargeter.py` (the facade), and `synthetic.py` (the operator and vehicle generators, in-package because they are the only way the vehicle case can be tested at all).

4.2 Driving it

The mode rides the existing `control_mode` wire message as an additive `frame` field, orthogonal to `region`: region scopes which *joints* the torque engine owns, frame scopes which *measurements* the retargeter may believe. It is absent-safe — an unrecognised or missing value falls back to LOCAL, because a moving base is the consequential state and must be asked for, never inherited from a silent or stale client. The Vision Pro app exposes it as a `[Local | Global]` segmented control beside Stiff/Soft, plus a console button reachable inside full immersion.

Switching is *live*: unlike the Wrist/WholeBody switch, where the two methods are different controllers and the outgoing one must be torn down, the two frames are the same controller reading different measurements, so the app neither stops tracking nor discards the calibration and the server makes the handoff continuous.

In the live engine the split was implemented by gating the body-follow channel at its source: in local mode the head-displacement cascade (base, torso, lift) was not evaluated at all, because every input it consumes is $\mathcal{F}_a$-referenced. The arms were untouched by the switch — they map through the operator's own body frame either way — which is what made the mode usable on the mature arm path without waiting for the new estimator. **That code was removed on 2026-07-28** (`set_retarget_frame`, `LocalBodyFollow`, the `frame` field of `control_mode`, and the app's LOCAL | GLOBAL segment); the live engine runs the global law only. The package below is unaffected — it never ran on the control thread.

4.3 Reproducing

```
python3 controller/experiments/exp_retarget_modes.py   # every figure + the table
python3 -m pytest controller/tests/test_retarget_modes.py -q
cd controller/docs && make
```

5 Limitations

1. **Uncommanded motion under simultaneous head yaw and pitch** (§3.5, 176 mm mean). This is the dominant open issue and it is structural, not a tuning failure: two hands do not carry enough information to track the head-to-torso rotation while the head moves freely. Resolving it needs a torso-mounted tracker, or a calibration that pins the shoulders far more tightly than anthropometric ratios do. Both are additive to this architecture.
2. **Accuracy at range-of-motion corners.** At simultaneous large yaw, pitch and roll the objective's global minimum is no longer the truth (mean 20°, worst 67° yaw error). The estimate stays bounded — it does not diverge or flip — but it is wrong.

3. **The reduction is exact only per snapshot.** (1) cancels $\mathcal{F}_a$ when the head and hand samples were resolved against the same anchor pose. ARKit resolves both from one snapshot, so the residual is whatever relocalisation occurs between snapshots; it is bounded but not zero, and is not separately measured here.
4. **The mode is no longer live.** As of 2026-07-28 the shipped engine offers global only, by operator decision, so everything this document claims about local mode is now claimed about an offline package rather than about the robot the apps drive. The vehicle case it was built for is therefore *unaddressed in the product*, not solved — a real gap, stated here rather than in a changelog.
5. **Two lift laws, while both existed.** Global keeps head-height, local used the comfort region, by product decision. The cross-fade kept a switch smooth, but it was two behaviours to maintain and the operator could feel the difference. Only the global law remains.
6. **No hardware verification.** Every number here is synthetic or simulated. The Vision Pro app builds and its tests pass, the controller suite passes, and §3.6 measures the modes end-to-end against the real wire — but against the *simulator*: nothing has been run on `thor`, and the visionOS simulator cannot exercise hand tracking at all, so the operator-side half of the loop is synthetic throughout.
7. **The new wrist-target map is not yet the live one.** The mode switch ships on the existing, operator-validated GMR arm map; the (12) implementation in this package is validated in simulation and reserved for an A/B against it.

References

[1] J. P. Araújo *et al.*, "Retargeting matters: General motion retargeting for humanoid motion tracking," *ICRA*, 2026.

[2] M. Shimizu *et al.*, "Analytical inverse kinematic computation for 7-DOF redundant manipulators with joint limits," *IEEE T-RO*, vol. 24, no. 5, 2008.

[3] X. Xu *et al.*, "HoMMI: Learning whole-body mobile manipulation from human demonstrations," 2025.

[4] Y. Jiang *et al.*, "AvatarPoser: Articulated full-body pose tracking from sparse motion sensing," *ECCV*, 2022.

[5] C. Lu *et al.*, "Mobile-TeleVision: Predictive motion priors for humanoid whole-body control," 2024.

[6] D. A. Winter, *Biomechanics and Motor Control of Human Movement*, 4th ed., Wiley, 2009.

[7] J. W. Youdas *et al.*, "Normal range of motion of the cervical spine: an initial goniometric study," *Phys. Ther.*, vol. 72, no. 11, 1992.

Related documents in this folder: `whole_body_retargeting.tex` (the shipped arm maps and the lazy followers), `body_frame.tex` (the global torso-frame estimator this package adapts), `calibration.tex` (the per-operator anthropometry), `mabel_controller.tex` (the teleop stack as a whole). Derivations are referenced rather than repeated.